

Statistics in RStudio

Anders Jensen

03/12/25

AndersR

Introduction

Working with data is central to research across all disciplines in the University, yet one of the biggest challenges people face is not only collecting data, but understanding how to analyse and interpret it. Many students and researchers learn statistics in a way that separates the underlying concepts from the practical tools used to apply them. Typically, courses focus heavily on the mathematical background first, and only later show how to carry out the analyses in software. While this approach works for some, it often leaves people feeling confident in neither the theory nor the implementation.

In this workbook we are taking a slightly different approach. While the background behind statistical tests will be mentioned we are more interested in teaching how to prepare your data so that statistical tests can be performed. Finally we also want to focus on how the analysis can be correctly reported. This guide assumes you already have some familiarity with R and RStudio and if this is not the case we recommend accessing the “Intro to R” and “Intro to GGplot2” handbooks available [here](#).

Statistics is most powerful when the concepts and the code are learned together, reinforcing each other. By working through real examples in R, you will build both an intuition for the underlying ideas and the practical skills to apply them in your own projects.

Key Advantages of Using R Programming for Statistics

- **Simplicity:** The R coding language was designed for statistics and developed by statisticians, making it optimal compared with other coding languages.
- **Reproducible:** Statistical analyses in R are fully script-based, making your workflow transparent, repeatable, and easy to audit.
- **Open-Source:** R and Rstudio are completely open-source and freely available to anyone, removing potential barriers seen in other software.
- **Range of tools:** R offers a wide-range of different tools to perform various statistical tests, which may be missing from other software. Newly developed statistical methods are also more likely to be implemented in R before other stats packages.

Statistics packages

One of the major advantages of using R for statistical analysis is the vast ecosystem of specialised packages available to researchers. These packages range from widely used, general-purpose tools to highly specific implementations created to address niche analytical needs. In practice, if there is a statistical method, model, or workflow you wish to apply, it is highly likely that someone in the research community has faced the same problem and developed an R package to solve it. Many of these packages are freely available on CRAN, and even more cutting-edge or experimental tools can be installed directly from GitHub, ensuring that researchers always have access to the latest methods.

Because of this it is impossible for me to discuss every stats package but I will briefly summarise two of the most common and the two that are used in this handbook.

The ‘stats’ package

The **stats** package is one of the core components of R and is installed automatically when you begin. Because of this, you do not need to load it manually and all the main functions are already present. This package contains many of the fundamental statistical tools that researchers rely on, including descriptive statistics, correlation methods, hypothesis tests (such as t-tests and ANOVA), linear and generalised linear models, non-parametric tests, probability distributions, and model diagnostics.

Most introductory and intermediate statistical analyses in R are built upon functions from the stats package. Even when you use higher-level packages, many of them serve as user-friendly wrappers around these base functions.

The ‘rstatix’ package

The **rstatix** package is designed to make statistical analysis in R more approachable, especially for users who prefer a tidy, pipe-based workflow. The pipe is a major aspect of the **dplyr** package and **rstatix** implements this to make a streamlined approach to statistical analysis. The package includes t-tests, ANOVA, effect size calculations, correlation, assumption checks, and post-hoc comparisons.

The way the code is written is consistent and intuitive, making it well suited for teaching or for researchers who want clear, concise code. Most of the case-studies presented in this document will utilise **rstatix** as it is clearer to understand the code, however a lot of the time this will come down to personal preference.

Choosing the correct test

Selecting the appropriate statistical test is a crucial step in any analysis. The choice depends not on preference but on the structure of your data and the type of question you are trying to answer. Factors such as the number of groups being compared, whether the data are paired or independent, the level of measurement, and whether assumptions like normality are met all play a role in determining which test is suitable.

As a general rule the following sequence of questions are really helpful in deciding the correct statistical test:

- 1) What is my research question?
- 2) What types of variables do I have?
- 3) How many different groups do I have?
- 4) Are the groups independent or paired? (Do they come from the same donors)
- 5) Is my data normally distributed?

There are other questions that are needed in more niche circumstances but if you have a clear answer to these five questions then you can almost always decide on an appropriate test.

Data cleaning

One of the major hurdles when performing statistical analysis in R is preparing your dataset so that it fits the requirements of the functions you plan to use. Many statistical errors in R are not caused by mistakes in the code, but by data that is structured incorrectly for the chosen analysis. Even if your analytical code is correct, an incompatible or inconsistent data format will prevent your functions from running properly.

In this section, we will cover the most common data-cleaning tasks that researchers need to perform. These include reshaping data between wide and long formats, transposing datasets, and creating summary tables. These skills form the foundation of effective statistical work in R and will be used repeatedly throughout the handout.

Wide to long

The best way to understand wide and long data is to think of a very simple example dataset and see how that data may be presented. Imagine the following dataset, you have different measurements for a large cohort of people (age, height, weight) and each row is a person as shown below.

```
##      Name Age Weight Height
## 1   John  36    180    160
## 2 Steven  38    160    150
## 3   Ryan  41    155    170
```

This is a classic example of wide data, but if we wanted to rearrange that to long data we would need to apply some simple R code.

So instead of our data being structured: Measurement - Var1 - Var2 - Var3

We are trying to move our data to: Measurement - Value - Variable

The easiest way to do this is with the **reshape2** package using the **melt()** function. This package is also pipe-friendly and can be used with or without the **dplyr** package.

```
# loading packages
library(reshape2)
library(dplyr)

# convert to long
df_long <- df %>%
  melt()

# visualise
head(df_long)
```

```
##      Name variable value
## 1   John      Age     36
## 2 Steven      Age     38
## 3   Ryan      Age     41
## 4   John    Weight    180
## 5 Steven    Weight    160
## 6   Ryan    Weight    155
```

So you should be able to follow the changes we have made to the dataset and how we have the same overall information just represented in a different way. Of course this transformation can be done manually but this takes a lot of time.

Long to wide

Sometimes, you may be in the opposite situation and from long to wide data. This transformation is less commonly required than going from wide to long, but it is useful in certain cases, such as creating tables or preparing data for specific analyses.

To do this we are going to use the `pivot_wider()` function from the **tidyr** package.

```
# load package
library(tidyr)

# go from long data back to wide data
df_wide <- df_long %>%
  pivot_wider(names_from = variable, # where the variables are stored (column name)
              values_from = value) # where the values are stored (column name)

# visualise
head(df_wide)
```

```
## # A tibble: 3 x 4
##   Name      Age Weight Height
##   <chr>   <dbl>   <dbl>   <dbl>
## 1 John     36     180     160
## 2 Steven   38     160     150
## 3 Ryan     41     155     170
```

If you ever run into difficulties doing this, the word *pivot* is commonly used to describe this process, whether that involves using AI to help or searching on stack overflow.

Transposing data

The next transformation that can be useful is transposing data, which involves switching the rows and columns so that rows become columns and columns become rows.

Using the initial dataframe as an example this can be achieved using the `t()` function as seen below:

```
# transpose
df_transposed <- df %>%
  t()

# convert to data frame
df_transposed_data <- df_transposed %>%
  as.data.frame()

# visualise
head(df_transposed_data)
```

```
##      V1      V2      V3
## Name  John Steven Ryan
## Age    36     38     41
## Weight 180    160    155
## Height 160    150    170
```

So you should see that the `t()` function is also pipe-friendly. You can also achieve this transformation without the pipe but you need to specify the dataframe within the function.

You can also see within the code snippet that using this function converts your dataframe into a matrix, in order to change this back you can use the `as.data.frame()` function.

Summarising data

There may be cases where you first need to work out means and standard deviations before analysing your data or just because you may want to calculate these values. Fortunately this is also simple with the **dplyr** packages and specifically two functions that work well together.

The two functions are `group_by()` and `summarise()`

These functions allow you to group by your categorical variables and then work out different descriptive statistics on the numeric variables.

Taking the previous example data, lets say we wanted to know the average height, weight and age of the participants in the study.

```
# process data
df_long <- df %>%
  melt()

# summarise data
summary_stats <- df_long %>%
  group_by(variable) %>% # first group them
  summarise(mean = mean(value), # work out mean
            sd = sd(value), # work out sd (standard deviation)
            max = max(value), # maximum value
            lowest_value_haha = min(value)) # minimum value (proof that the name doesnt matter)

# visualise
head(summary_stats)
```

```
## # A tibble: 3 x 5
##   variable    mean     sd    max lowest_value_haha
##   <fct>    <dbl> <dbl> <dbl>          <dbl>
## 1 Age      38.3  2.52   41             36
## 2 Weight  165  13.2  180            155
## 3 Height  160   10   170            150
```

This provides you with descriptive information about your data and can also be used to calculate summary statistics, such as the mean, which may be required before performing certain statistical analyses.

Important note: If you have missing values in your data (NA) use `na.rm = T` after the word value (`mean = mean(value, na.rm = T)`), otherwise the calculation fails.

Summary of Statistical Tests

Overview of common tests, their data types, examples, and assumptions

Test	Type of Data	Example Use	Assumptions	Function
Normality testing	Continuous data	Distribution of heights in people from England	Independent observations	<code>shapiro.test()</code>
T-Test	Continuous data from two groups	Comparing the heights of two breeds of dogs	Normality, independence, equal variance	<code>t.test()</code>
ANOVA	Continuous data from more than two groups	Analysing income between different age groups	Normality, independence, equal variance	<code>aov()</code>
Correlation	Two continuous variables	Assessing link between weight and height	Linearity, normality	<code>cor.test()</code>
Linear regression	Continuous outcome variable	Using height to predict weight	Linearity, normality of residuals, homoscedasticity	<code>lm()</code>
Logistic regression	Binary outcome variable	Comparing exam performance on application success	Linearity, independence	<code>glm()</code>
Chi-square	Categorical count data	Seeing if a dice is biased towards rolling sixes	Expected count > 5, independence	<code>chisq.test()</code>

Statistical tests

In this section some of the most common statistical tests will be covered and each test will be divided into three sections.

- 1) When to do the test? Including examples
- 2) How to code the test?
- 3) How to interpret and report the results?
- 4) Are there any other examples?

All of the data sets used in the examples can be accessed in RStudio, to learn more information about the data simply run `?iris`, obviously changing the word `iris` for the name of the data used.

Below is a brief summary table of the tests discussed in the next few sections.

Normality testing

Normality testing is used to determine whether a continuous variable follows a normal distribution (bell-shaped curve), which is an important assumption for many statistical tests.

When should I do this?

You should run some kind of normality test before running a parametric statistical test that assume normal data.

Examples

- 1) Seeing if the distribution of heights in a group of people is normal.
- 2) Determining if the temperature throughout the year is normally distributed.

How do I code this?

For this example we are using the iris dataset. I want to check if the sepal length of the three different flower types follows a normal distribution.

The code below uses both the **dplyr** package as well as the **rstatix** package. The **ggplot2** package was used to visualise the data.

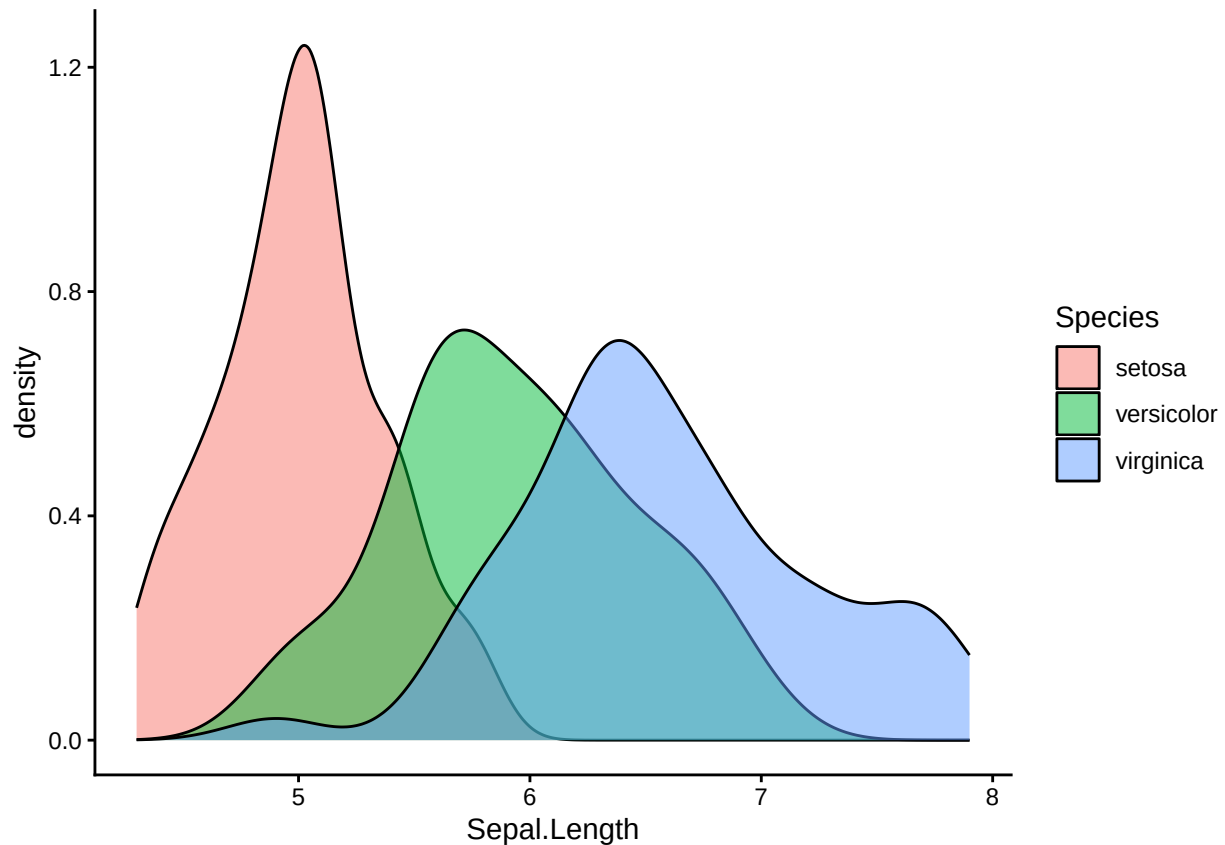
```
# packages
library(dplyr)
library(rstatix)
library(ggplot2)

# load the data
data(iris)

# perform shapiro tests
iris %>%
  group_by(Species) %>%
  shapiro_test(Sepal.Length)
```

```
## # A tibble: 3 x 4
##   Species    variable    statistic      p
##   <fct>      <chr>          <dbl> <dbl>
## 1 setosa    Sepal.Length    0.978 0.460
## 2 versicolor Sepal.Length    0.978 0.465
## 3 virginica Sepal.Length    0.971 0.258
```

```
# plot the data
iris %>%
  ggplot(aes(x = Sepal.Length))+
  geom_density(aes(fill = Species),
    alpha = 0.5)+
  theme_classic()
```



How do I report this?

So the result shows that all three species have a normal distribution because the test is against the normal distribution and the p-value is > 0.05 .

“Sepal length measurements for Setosa, Versicolor and Virgnica ($n = 50$ per group) did not significantly deviate from a normal distribution, as indicated by Shapiro-Wilk test ($p > 0.05$)”

Other examples

Other types of normality test includes the Kolmogorov-Smirnov test (*ks.test*) and the Anderson-Darling (*ad.test*) (**nortest package**). Generally these tests are used in larger sample size studies, so anything above $n = 300$ these tests should be considered.

T-Test

A t-test is used to compare the means of two groups to determine whether they are statistically different from one another. It is one of the most common statistical tests in research and is used when the outcome variable is continuous.

When should I do this?

You should use a t-test when you want to compare two groups on a continuous variable. There are two main types; paired and unpaired based on the independence of samples.

Examples

- 1) Comparing blood pressure between males and females (unpaired)
- 2) Measuring reaction time before and after caffeine (paired)

How do I code this?

For this example we are using the mtcars dataset. I want to check if the miles per gallon differs between straight and v-shaped engines.

The code below uses both the **dplyr** package as well as the **rstatix** package. The **ggplot2** package was used to visualise the data.

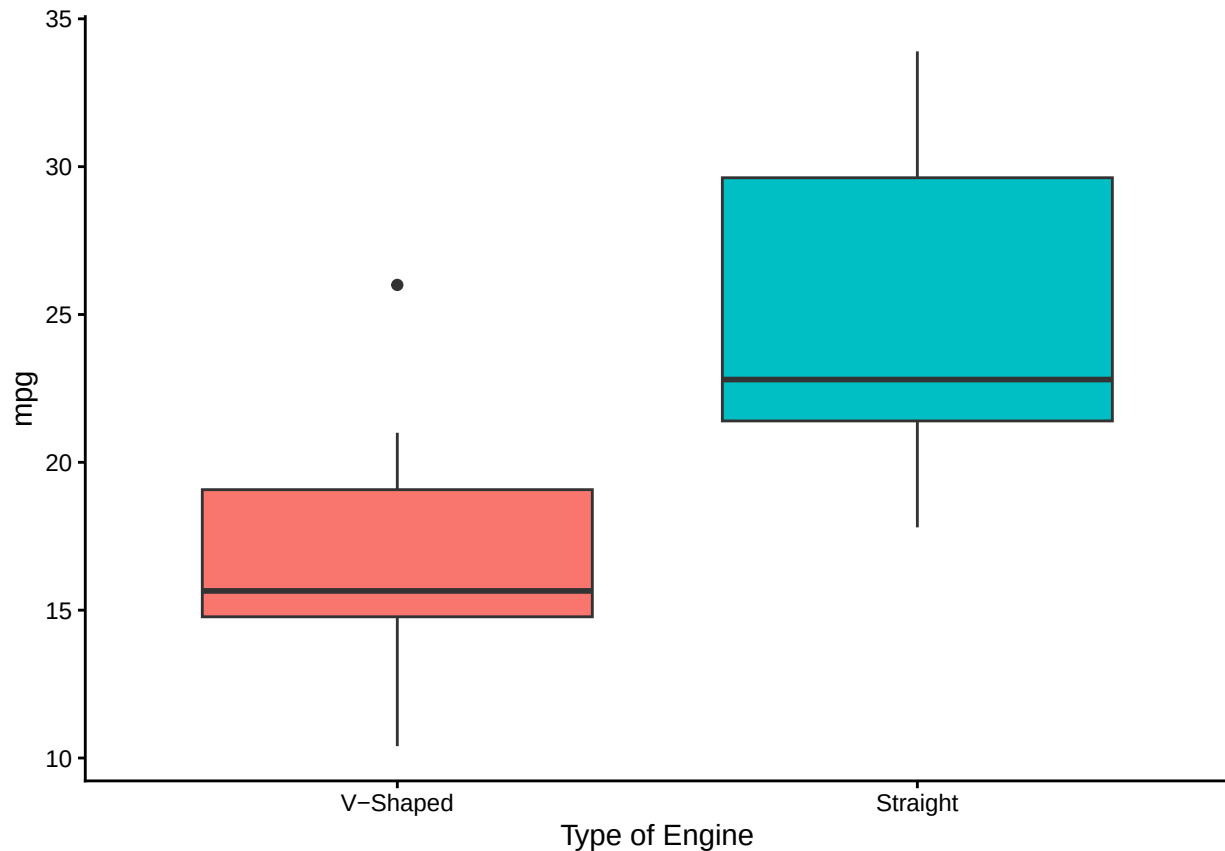
```
# packages
library(dplyr)
library(rstatix)
library(ggplot2)

# load data
data(mtcars)

# perform test
mtcars %>%
  t_test(mpg~vs)
```

```
## # A tibble: 1 x 8
##   .y.   group1 group2    n1    n2 statistic    df      p
## * <chr> <chr>  <chr> <int> <int>    <dbl> <dbl>  <dbl>
## 1 mpg     0      1      18    14    -4.67  22.7  0.00011
```

```
# visualise
mtcars %>%
  ggplot(aes(x = as.factor(vs), y = mpg))+
  geom_boxplot(aes(fill = as.factor(vs)), show.legend = F)+
  scale_x_discrete(labels = c("V-Shaped", "Straight"))+
  theme_classic()+
  labs(x = "Type of Engine")
```



How do I report this?

So as you can see from the table we have a p-value that is below 0.05 and therefore based on this threshold we can conclude that the difference in miles per gallon between V-Shaped and Straight engines is significantly different.

An independent samples t-test showed that vehicles with straight engines had significantly higher MPG than those with V-shaped engines ($p < 0.001$).

Other examples

A paired t-test is used when the same participants (or matched pairs) are measured twice, for example, before and after an intervention, or when each subject experiences two conditions. This test accounts for the fact that the observations are not independent. To do this kind of test you would use the (`paired = T`) argument in the code above.

A one-sample t-test is used when you want to compare the mean of a single sample to a known or hypothesised population mean. To do this you would specify the value using the (`mu = x`) argument in the code above.

Finally a Mann-Whitney U test (Wilcoxon rank-sum) can be used when comparing two independent groups when assumptions of the t-test (normality) are not met. The code for this is very similar but instead of `t_test()` function you can use `wilcoxon_test()`.

ANOVA

ANOVA (Analysis of Variance) is used to compare the means of three or more groups to determine whether there is a statistically significant difference between them. It is one of the most widely used tests when the outcome variable is continuous and the predictor is categorical with at least three levels.

When should I do this?

You should use ANOVA when you want to compare three or more groups on a continuous outcome.

If your ANOVA is significant, you typically follow up with post-hoc tests to determine which specific groups differ.

Examples

- 1) Comparing working hours across the working week
- 2) Measuring marathon times among different age groups (Young, Middle, Old)

How do I code this?

For this example we will use the `InsectSprays` dataset. I want to see which of the sprays is the most effective and if there is a significant difference.

The code below uses both the **dplyr** package as well as the **rstatix** package. The **ggplot2** package was used to visualise the data.

```
# packages
library(dplyr)
library(ggplot2)
library(rstatix)

# load data
data(InsectSprays)

# perform anova
InsectSprays %>%
  anova_test(count~spray)
```

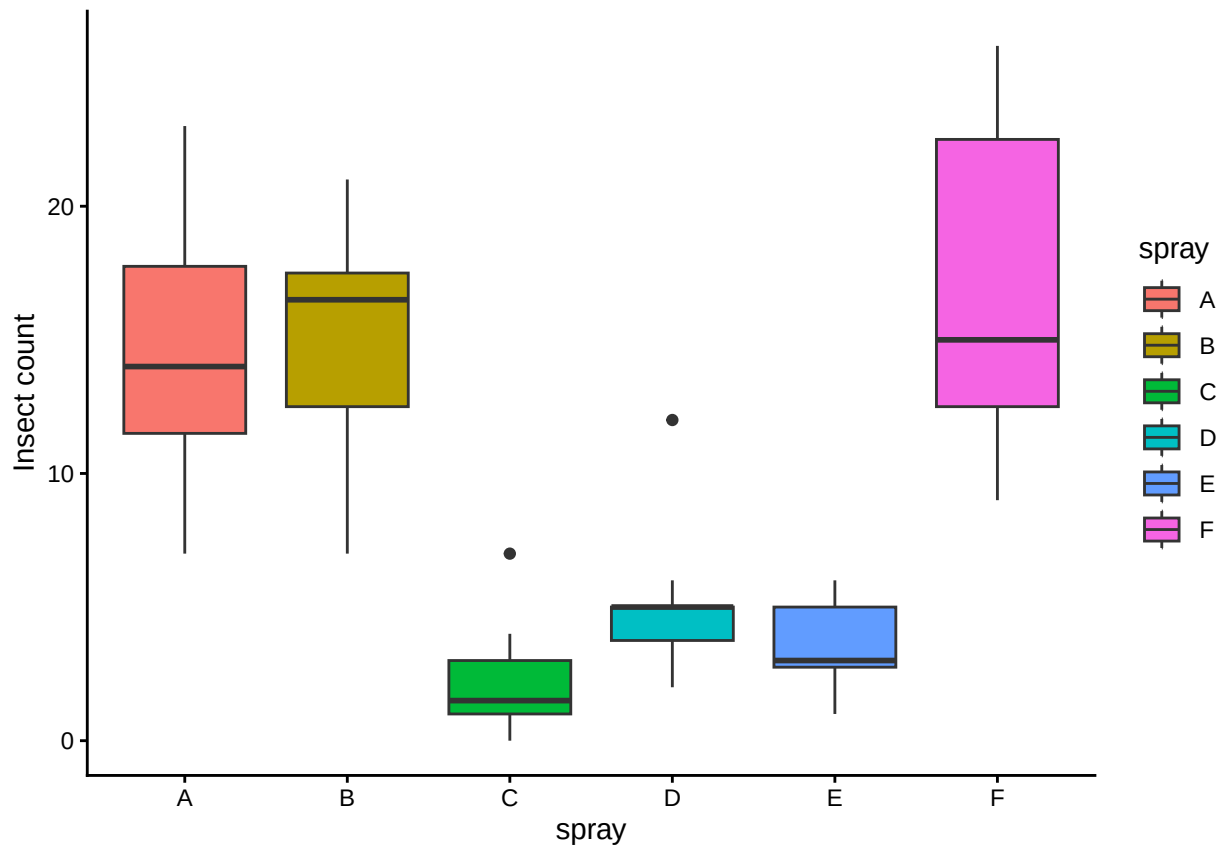
```
## ANOVA Table (type II tests)
##
##   Effect DFn DFd      F      p p<.05    ges
## 1  spray   5  66 34.702 3.18e-17    * 0.724
```

```
# perform post-hoc analysis as the result is significant
InsectSprays %>%
  tukey_hsd(count~spray)
```

```
## # A tibble: 15 x 9
##   term  group1 group2 null.value estimate conf.low conf.high  p.adj
## * <chr> <chr> <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 spray A      B          0    0.833    -3.87    5.53 9.95e- 1
## 2 spray A      C          0   -12.4   -17.1   -7.72 1.08e- 9
```

```
## 3 spray A      D      0 -9.58 -14.3 -4.88 1.44e- 6
## 4 spray A      E      0 -11    -15.7 -6.30 4.09e- 8
## 5 spray A      F      0  2.17  -2.53  6.87 7.54e- 1
## 6 spray B      C      0 -13.3 -17.9 -8.55 1.18e-10
## 7 spray B      D      0 -10.4 -15.1 -5.72 1.79e- 7
## 8 spray B      E      0 -11.8 -16.5 -7.13 4.84e- 9
## 9 spray B      F      0  1.33  -3.37  6.03 9.6 e- 1
## 10 spray C     D      0  2.83  -1.87  7.53 4.92e- 1
## 11 spray C     E      0  1.42  -3.28  6.12 9.49e- 1
## 12 spray C     F      0 14.6    9.88 19.3 0
## 13 spray D     E      0 -1.42  -6.12  3.28 9.49e- 1
## 14 spray D     F      0 11.7    7.05 16.4 6 e- 9
## 15 spray E     F      0 13.2    8.47 17.9 1.49e-10
## # i 1 more variable: p.adj.signif <chr>
```

```
# visualise data
InsectSprays %>%
  ggplot(aes(x = spray, y = count))+
  geom_boxplot(aes(fill = spray))+
  theme_classic()+
  labs(y = "Insect count")
```



How do I report this?

The results show that generally across the dataset there is a significant difference in the insect count based on the type of spray as the p-value from the anova test was less than 0.05. Furthermore, we can see that the main differences come when comparing ABF against CDE.

A one-way ANOVA showed a significant effect of spray type on insect count ($p < 0.05$). Post-hoc Tukey tests indicated that sprays A, B, and F did not differ significantly from one another ($p_{\text{adj}} > 0.05$), and sprays C, D, and E also did not differ significantly within their group ($p_{\text{adj}} > 0.05$). However, all comparisons between sprays in the ABF group versus those in the CDE group were statistically significant ($p_{\text{adj}} < 0.0001$), with sprays A, B, and F producing higher insect counts than sprays C, D, and E.

Other examples

A two-way ANOVA is used when you want to examine the effect of two independent variables on a continuous outcome at the same time. This test also allows you to explore whether there is an interaction effect, meaning the effect of one variable depends on the level of the other. To code this simply add the second variable into the code above `anova_test(continuous ~ var1 * var2)`.

A Kruskal–Wallis test is a non-parametric alternative to one-way ANOVA and is used when the assumptions of ANOVA (such as normality or equal variances) are not met. It compares the distribution of three or more independent groups and is based on ranked data rather than means. To code this you could change the `anova_test()` function for the `kruskal_test()` function. Then instead of the `tukey_hsd()` function you would use the `dunn_test()` function.

Correlation

Correlation is used to measure the strength and direction of the relationship between two continuous variables. The result is expressed as a correlation coefficient, which ranges from -1 to $+1$, where values closer to these extremes indicate stronger relationships.

When should I do this?

You should use a correlation when:

You want to understand whether two continuous variables move together and in cases where there aren't clear independent and dependent variables.

Examples

- 1) Is there an association between temperature and ice cream sales?
- 2) Examining the relationship between height and the number of points scored in basketball.

How do I code this?

For this example we will use the `airquality` dataset. I want to see if temperature is associated with ozone concentration.

The code below uses both the `dplyr` package as well as the `rstatix` package. The `ggplot2` package was used to visualise the data.

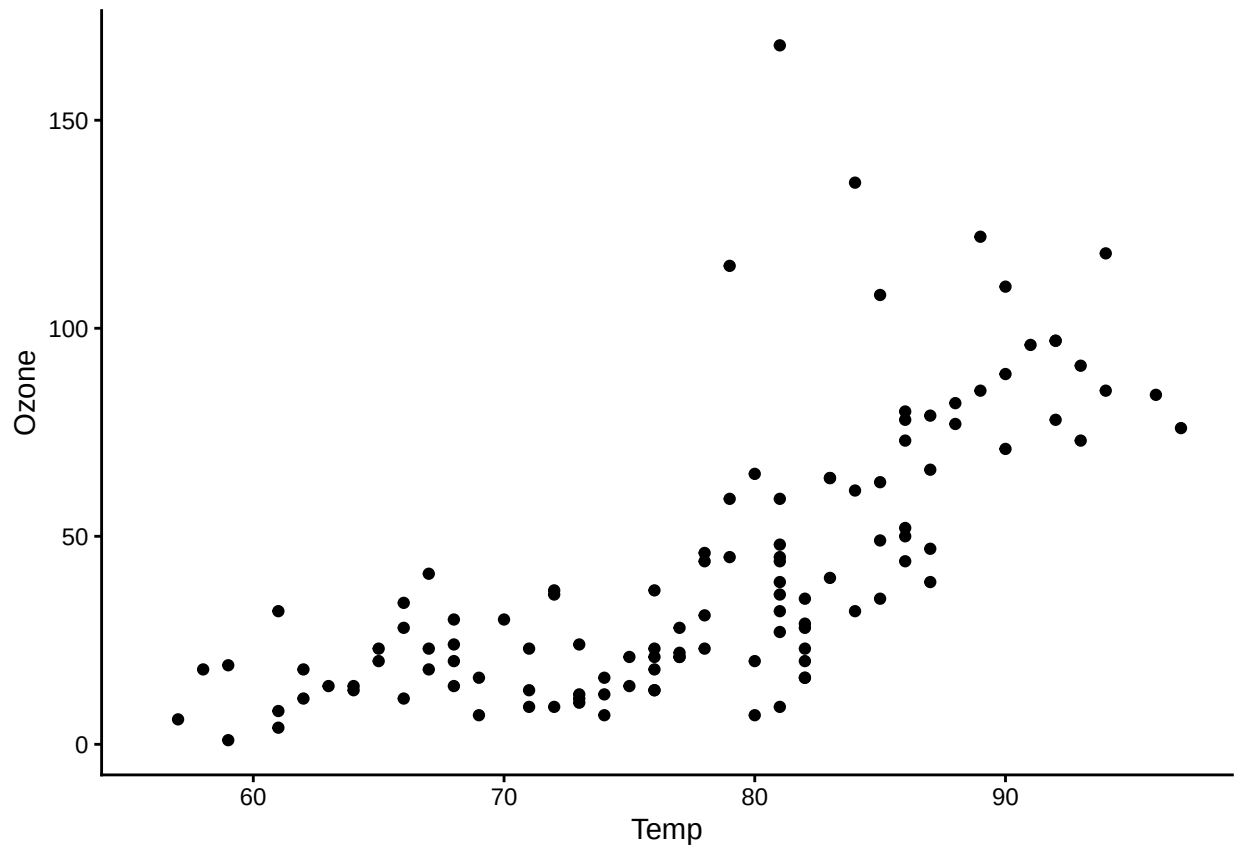
```
# load packages
library(dplyr)
library(rstatix)
library(ggplot2)

# load data
data(airquality)

# run correlation
airquality %>%
  cor_test(vars = Temp, vars2 = Ozone, method = "pearson")
```

```
## # A tibble: 1 x 8
##   var1  var2    cor statistic      p conf.low conf.high method
##   <chr> <chr> <dbl>    <dbl>    <dbl>    <dbl>    <dbl> <chr>
## 1 Temp  Ozone   0.7      10.4 2.93e-18   0.591    0.781 Pearson
```

```
# visualise data
airquality %>%
  ggplot(aes(x = Temp, y = Ozone))+
  geom_point()+
  theme_classic()
```

How do I report this?

So we can see from the result that there is a significant positive association between temperature and ozone concentration.

A Pearson correlation showed there was a significant positive correlation between temperature and ozone concentration ($r = 0.70$, $p < 0.0001$)

Other examples

The main other example is a Spearman's rank test, which is used when the assumptions of correlation (such as normality) are not met. Very simply change the method in the above code to say "spearman" instead of pearson in these cases.

Chi-square

A Chi-square test is used to determine whether there is an association between two categorical variables. It compares the observed frequencies in each category with the frequencies that would be expected if there were no relationship between the variables.

The result is expressed as a Chi-square statistic (χ^2) along with a p-value, which indicates whether the association between variables is statistically significant.

When should I do this?

You should use a Chi-square test when: You want to examine whether two categorical variables are associated with one another.

Examples

- 1) Is there an association between smoking status (smoker/non-smoker) and the presence of lung disease (yes/no)?
- 2) Is there a relationship between gender and preferred type of exercise (running, swimming, cycling)?

How do I code this?

For this example we will use the HairEyeColor dataset. This dataset contains counts of people by hair color, eye color, and sex

```
# load packages
library(dplyr)
library(ggplot2)

# load data
data("HairEyeColor")

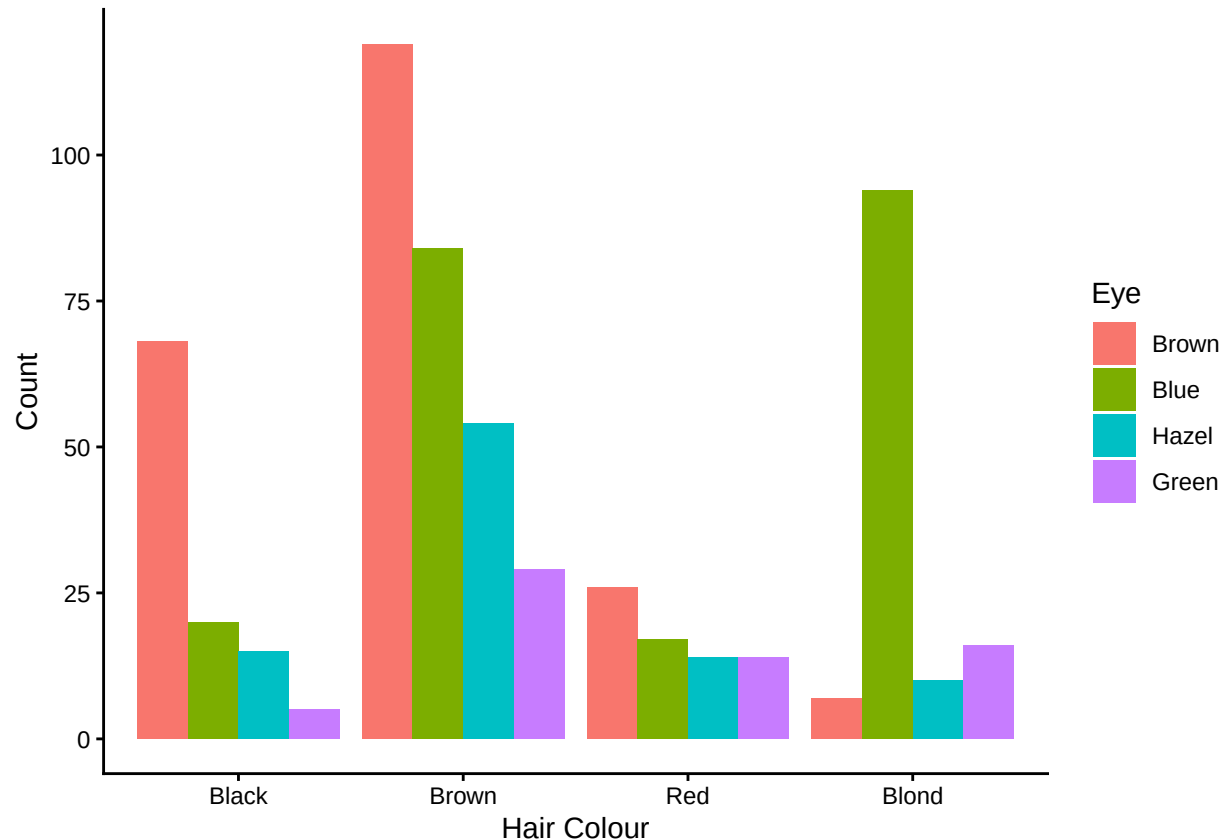
# Merge male/female
df <- margin.table(HairEyeColor, c(1,2)) # easy function to merge

# run chi-square
chi_square <- chisq.test(df)
chi_square
```

```
##
## Pearson's Chi-squared test
##
## data: df
## X-squared = 138.29, df = 9, p-value < 2.2e-16
```

```
# Plot counts
df %>%
  as.data.frame() %>%
  ggplot(aes(x = Hair, y = Freq)) +
  geom_bar(stat = "identity",
           position = "dodge",
```

```
aes(fill = Eye)) +  
labs(x = "Hair Colour",  
     y = "Count") +  
theme_classic()
```



How do I report this?

From the output, we can determine whether there is a statistically significant association between hair color and eye color.

A Chi-square test of independence showed that there was a significant association between hair color and eye color ($X^2(df) = 592$, $p < 0.001$).

Other examples

If the expected counts in any category are very low (less than 5), you can use Fisher's Exact Test instead. You can also include the third variable, Sex, to explore whether associations differ by gender.

Linear regression

Linear regression is used to examine the relationship between a continuous outcome variable (dependent variable) and one or more continuous or categorical predictor variables (independent variables). The goal is to model how changes in the predictors are associated with changes in the outcome.

It differs slightly from correlation because linear regression examines how one variable can be used to predict another, whereas correlation simply measures the strength and direction of the relationship between two variables without defining a predictor or outcome.

When should I do this?

You want to predict or explain a continuous outcome from one or more predictor variables and the relationship is expected to be approximately linear.

Examples

- 1) Predicting house prices based on house size
- 2) Investigating if exercise duration predicts the number of calories burned

How do I code this?

For this example, we will use the mtcars dataset. We will predict mpg (miles per gallon) from wt (car weight).

```
# load packages
library(dplyr)
library(ggplot2)

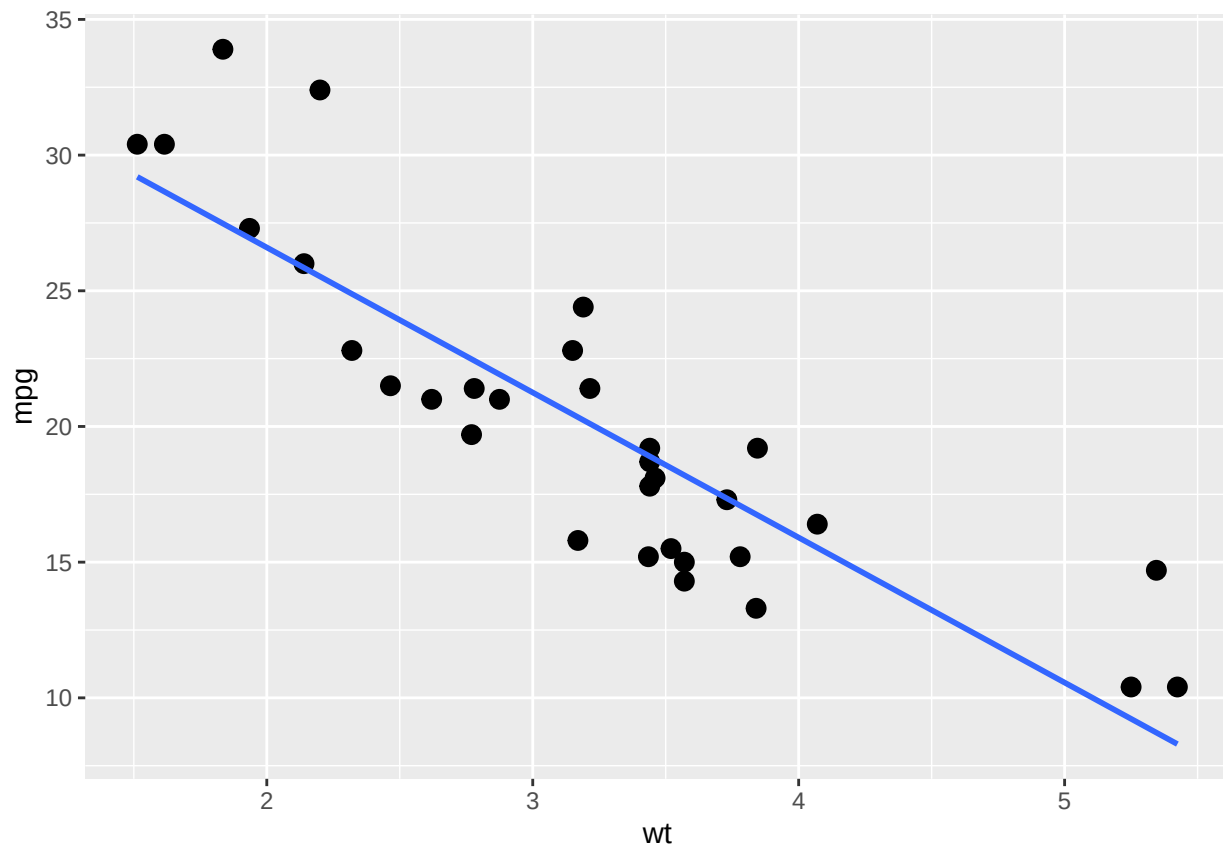
# load dataset
data("mtcars")

# Fit linear regression model: mpg ~ wt + hp
lm_model <- lm(mpg ~ wt, data = mtcars)
summary(lm_model)

##
## Call:
## lm(formula = mpg ~ wt, data = mtcars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.5432 -2.3647 -0.1252  1.4096  6.8727
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   37.2851     1.8776  19.858 < 2e-16 ***
## wt           -5.3445     0.5591  -9.559 1.29e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 3.046 on 30 degrees of freedom
## Multiple R-squared:  0.7528, Adjusted R-squared:  0.7446
## F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10
```

```
# Visualise relationship
mtcars %>%
  ggplot(aes(x = wt, y = mpg))+
  geom_point(size = 3)+
  geom_smooth(method = "lm", se = F)
```



How do I report this?

From the output, you can see the coefficients for each predictor, their significance, and the overall model fit. A linear regression showed that weight significantly predicts miles per gallon. Heavier cars have a higher miles per gallon ($p < 0.0001$). The model explains 74% of variance in mpg ($R^2 = 0.74$)

Other examples

Linear regression can also be extended in several ways depending on the type and number of predictors.

Multiple linear regression can be used when you want to include more than one predictor variable in the model. For example, predicting house prices based on property size, number of bedrooms, and year built.

Categorical predictors can also be included in a regression model by converting them to factors. For example, examining whether fuel efficiency differs between transmission types (automatic vs manual) while also accounting for engine size.

Further information

The aim of this book was to give you an introduction to statistics in RStudio and to hopefully improve your confidence in choosing the correct statistical tests and how to report them.

When applying these principles to your own data make sure the data is in the same structure as the examples used in this booklet.

If you have any advice on how to improve this booklet or anything that should be included, please do not hesitate to get in touch (hlajens2@liverpool.ac.uk).